

INDEPENDENT POST-REMEDIATION ASSESSMENT

andrewyng/openworker OpenWorker Code Intelligence Report

Prioritized source, dependency, workflow, architecture, maintainability, and repository-governance evidence with explicit confidence, applicability, remediation, verification, and claim boundaries.



EXACT TARGET

bbb806ce5453 · HEAD

EVIDENCE DISCIPLINE

100% traceable finding
coverage

REPORT CONFIDENCE

100/100 · high

441 analyzed files

79,827 code LOC

12 findings

clean working tree · fetched history

OVERALL SCORE

92/100

Strong

SECURITY

100/100

source + dependency + workflow
signals

EFFORT RANGE

12–29 days

engineering-only planning range

FINANCIAL IMPACT

Not modeled

A defensible dollar estimate requires organization-specific labor rates, incident history, delivery volume, and change-failure data.

Executive Summary

What the Evidence Says

The repository scores 92/100 (Strong) with high report confidence (100/100).

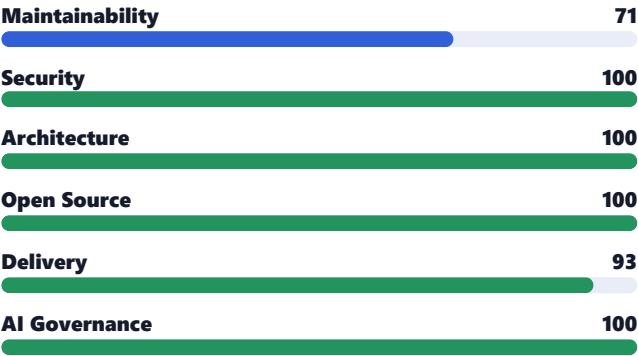
100% of findings contain traceable rule and source or repository evidence; 0 have a confirmed owner and 12 remain unassigned.

No dependency vulnerability rows were surfaced; package-manager audit status and lockfile evidence determine whether this is a verified clean result or a coverage gap.

Repository evidence mapping contains no material gap categories, but organizational process and operating effectiveness remain outside this scan.

The applicable critical/high/medium queue implies roughly 12–29 engineering days before discovery, release coordination, and production validation.

Score Dimensions



EVIDENCE COVERAGE

100%
12/12 findings

CONFIRMED OWNERS

0
12 unassigned

DEPENDENCY VULNS

0
0 critical · 0 high

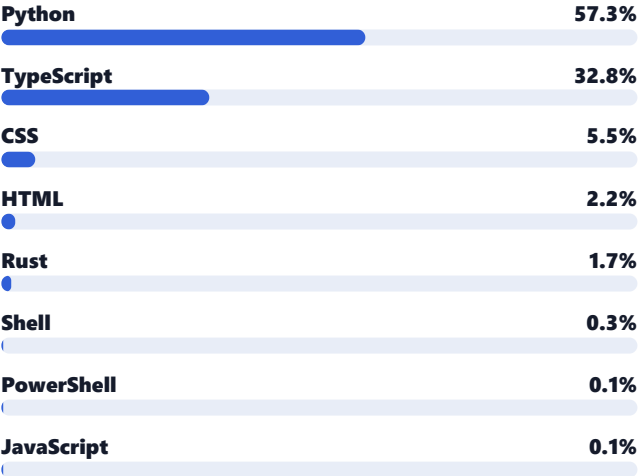
WORKFLOW GAPS

0
mutable action refs + implicit permission policies

Severity / Effort Distribution

	S	M	L	XL
CRITICAL	0	0	0	0
HIGH	0	0	6	0
MEDIUM	0	1	1	0
LOW	2	2	0	0

Codebase Composition



The overall score is a prioritization model. It is not a certification, exploitability determination, valuation opinion, productivity measurement, or substitute for runtime testing and human review.

Methodology, Scope, and Claim Boundaries

Exact Audit Target

Repository: <https://github.com/andrewyng/openworker>
Commit: [bbb806ce5453335d64ef163f9d5586544398c7e8](#) (July 25, 2026)
Branch: HEAD
Snapshot state: working tree clean; fetched history.
File discovery: Git tracked and non-ignored untracked files.
Generated: July 25, 2026; scanner runtime 44,928 ms.

Coverage and Traceability

SCAN COVERAGE

90.2%

441/489 candidate files

CODE LOC

79,827

86,941 analyzed text lines

EVIDENCE COVERAGE

100%

12 findings with traceable evidence

REPORT CONFIDENCE

100 / 100

high confidence

Repository Scope Classification

SCOPE	FILES	HOW FINDINGS ARE INTERPRETED
Production	269	Directly applicable to shipped or runtime code.
Build / delivery control	6	Applies to CI, release, packaging, or infrastructure controls.
Tests and fixtures	156	Contextual evidence; severity is reduced unless production reachability is established.
Examples and documentation	10	Advisory or informational; excluded from direct production-risk claims.
Generated report artifacts	0	Excluded from the report's own finding evidence to prevent self-contamination.

Analysis Procedure

- Enumerate text candidates with size, directory, ignore-file, and generated-artifact limits.
- Classify production, control-plane, test, fixture, example, documentation, and generated scope.
- Extract language-aware units and rule evidence; attach rule IDs, fingerprints, confidence, applicability, remediation, and verification.
- Inspect dependency manifests, lock evidence, package-manager audit output, workflow permissions, and immutable action provenance.
- Score with confidence/applicability weighting and diminishing penalties; do not infer dollar impact or historical trends from a single scan.

Limitations

Static repository analysis cannot prove runtime exploitability, production reachability, or control operation.

Dependency audit coverage depends on available package-manager tooling, network access, and resolvable manifests.

Git history metrics reflect the fetched history window, not necessarily the repository's full lifetime.

Framework mappings are evidence-alignment aids, not certification, attestation, or compliance determinations.

Dependency audit status: 2/2 applicable audit command(s) completed.

Project Profile and Stack Coverage

Detected Project Shape

The scan detected 441 analyzed files, 79,827 lines of code, 4 dependency or build manifest(s), and 8 language signal(s).

JavaScript/TypeScript application or package workspace

Python service, automation, or data/AI codebase

Rust systems, CLI, or backend component

multi-component repository requiring ownership mapping

This page tunes the audit interpretation by stack. A Node/TypeScript repo needs different evidence than a Go service, Ruby app, Rust CLI, JVM service, or .NET system. The remediation plan should preserve proof that fits the language and package ecosystem actually present.

Coverage Signals

ECOSYSTEMS

3

Python, Node.js, Rust

MANIFESTS

4

dependency/build files

LOCK GAPS

0

manifest(s) without lock evidence

LANGUAGES

8

detected code families

Language-Specific Review Focus

LANGUAGE	DEPENDENCY EVIDENCE	AUDIT FOCUS
Python 57.3% 45,764 LOC	Python, Node.js, Rust	requirements/pyproject reproducibility, runtime pinning, data/AI workflow validation, and test evidence.
TypeScript 32.8% 26,180 LOC	Python, Node.js, Rust	package lock integrity, type-safety boundaries, SDK/API drift, test evidence, and dependency reachability.
CSS 5.5% 4,353 LOC	Python, Node.js, Rust	Maintainability, ownership, dependency evidence, and test confidence for this stack.
HTML 2.2% 1,768 LOC	Python, Node.js, Rust	Maintainability, ownership, dependency evidence, and test confidence for this stack.
Rust 1.7% 1,333 LOC	Python, Node.js, Rust	Cargo.lock policy, crate ownership, unsafe/native boundaries, and release confidence.
Shell 0.3% 276 LOC	Python, Node.js, Rust	Maintainability, ownership, dependency evidence, and test confidence for this stack.

Manifest and Reproducibility Evidence

MANIFEST	ECOSYSTEM	DEPENDENCIES	REPRODUCIBILITY EVIDENCE
pyproject.toml	Python	6	lock/equivalent present
surfaces/gui/package.json	Node.js	21	lock/equivalent present
surfaces/gui/src-tauri/Cargo.toml	Rust	10	lock/equivalent present
stt/Cargo.toml	Rust	5	lock/equivalent present

Priority Action Plan

PRIORITY QUEUE
6
highest-confidence applicable findings

EFFORT RANGE
12–29 days
engineering-only estimate

NAMED OWNERS
0
12 findings remain unassigned

VERIFICATION SPECS
100%
remediation plus verification present

Recommended Sequencing

- Contain:** patch exploitable dependencies and tighten delivery controls before broad refactoring.
- Reduce change risk:** split the highest-complexity production units behind tests and stable interfaces.
- Institutionalize:** assign real owners, record acceptance criteria, and retain verification evidence from the next scan.

The effort range is a planning aid derived from finding size and applicability. It excludes discovery, release windows, stakeholder coordination, and production validation.

Highest-Priority Backlog

RISK	EVIDENCE	REQUIRED CHANGE	PROOF OF COMPLETION
HIGH P1 · L effort high confidence	Function complexity hotspot: App surfaces/gui/src/App.tsx:196 clearcode.maintainability.function-complexity-hotspot-app · direct App spans 1691 non-empty lines with 402 decision token(s) and a 75.1/100 modeled complexity score.	Split App around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path.	Run the affected test suite and re-scan surfaces/gui/src/App.tsx:196 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH P2 · L effort high confidence	Function complexity hotspot: Composer surfaces/gui/src/components/Composer.tsx:82 clearcode.maintainability.function-complexity-hotspot-composer · direct Composer spans 434 non-empty lines with 144 decision token(s) and a 60.8/100 modeled complexity score.	Split Composer around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path.	Run the affected test suite and re-scan surfaces/gui/src/components/Composer.tsx:82 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH P3 · L effort high confidence	Function complexity hotspot: Sidebar surfaces/gui/src/components/Sidebar.tsx:104 clearcode.maintainability.function-complexity-hotspot-sidebar · direct Sidebar spans 1020 non-empty lines with 204 decision token(s) and a 69.9/100 modeled complexity score.	Split Sidebar around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path.	Run the affected test suite and re-scan surfaces/gui/src/components/Sidebar.tsx:104 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH P4 · L effort high confidence	Large file hotspot surfaces/gui/src/App.tsx:1 clearcode.maintainability.large-file-hotspot · direct 1969 non-empty lines make ownership, testing, and refactoring harder.	Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.	Re-run the scan and review surfaces/gui/src/App.tsx:1 for the corrected pattern.
HIGH P5 · L effort high confidence	Large file hotspot coworker/server/app.py:1 clearcode.maintainability.large-file-hotspot · direct 1748 non-empty lines make ownership, testing, and refactoring harder.	Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.	Re-run the scan and review coworker/server/app.py:1 for the corrected pattern.
HIGH P6 · L effort high confidence	Large file hotspot coworker/server/manager.py:1 clearcode.maintainability.large-file-hotspot · direct 3490 non-empty lines make ownership, testing, and refactoring harder.	Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.	Re-run the scan and review coworker/server/manager.py:1 for the corrected pattern.

Security, Dependencies, and Delivery Controls

Current Risk Baseline

No historical trend series was available in this scan payload, so this page treats the current scan as the baseline for the next remediation cycle.

The baseline is still actionable: 12 unassigned finding(s), 0 repository-governance gap(s), and 0 framework evidence gap category/categories.

The applicable queue implies 12–29 engineering days. The next comparable scan should show finding closure, retained evidence, assigned ownership, and verified dependency/workflow controls.

Risk Signals to Track

SIGNAL	CURRENT VALUE	WHY IT MATTERS
Maintainability debt	71 / 100 medium	Prioritize high-risk functions and large files, then convert repetitive patterns into named helpers.
Ownership concentration debt	90% top contributor share medium	Document ownership, rotate reviews, and move implicit operational knowledge into runbooks.
Security debt	100 / 100 low	Resolve critical/high issues first, preserve audit output, and document residual accepted risk.
Architecture debt	100 / 100 low	Break cyclic dependencies, isolate cross-cutting modules, and define ownership boundaries.
Dependency governance debt	42 dependencies low	Pin production dependencies, keep lockfiles, record licenses, and enable dependency update automation.

DEPENDENCIES

42

4 manifests

RISKY SPECS

0

unpinned or loose versions

SECURITY FINDINGS

0

0 critical, 0 high

FIXABLE VULNS

0

package-manager remediable

Dependency Audit Evidence

SOURCE	STATUS / SEVERITY	PACKAGE	EVIDENCE
npm-audit	ok	surfaces/gui/package.json	Dependency governance signal from scan.
pip-audit	ok	pyproject.toml	Dependency governance signal from scan.

GitHub Actions Security Evidence

Workflow permissions, trigger privilege, expression interpolation, and action-reference provenance are evaluated independently from application source. Mutable tags and implicit permission defaults are supply-chain control gaps even when no exploit path is confirmed by static analysis.

GitHub Actions Security Evidence

WORKFLOWS
2
analyzed YAML files

EXPLICIT PERMISSIONS
2
0 implicit

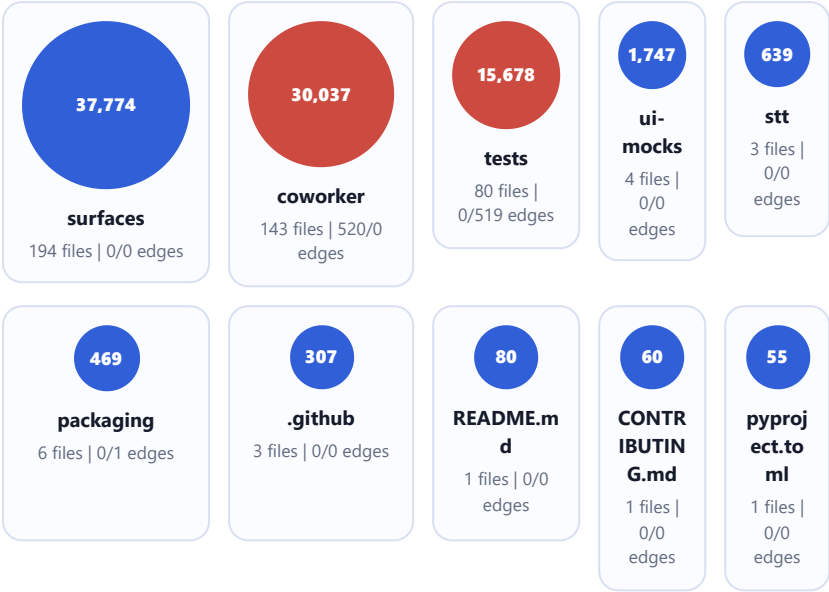
IMMUTABLE ACTIONS
15
0 mutable refs

PRIVILEGED PR
0
pull_request_target workflows

WORKFLOW	PERMISSIONS	ACTION PROVENANCE	PRIVILEGED / INJECTION SIGNALS
.github/workflows/ci.yml	explicit	6 immutable / 0 mutable	none detected
.github/workflows/release.yml	explicit	9 immutable / 0 mutable	none detected

Architecture and Maintainability

Architecture Map



Maintainability Narrative

The repository scores 92/100 (Strong) with high report confidence (100/100).

100% of findings contain traceable rule and source or repository evidence; 0 have a confirmed owner and 12 remain unassigned.

No dependency vulnerability rows were surfaced; package-manager audit status and lockfile evidence determine whether this is a verified clean result or a coverage gap.

Repository evidence mapping contains no material gap categories, but organizational process and operating effectiveness remain outside this scan.

The applicable critical/high/medium queue implies roughly 12–29 engineering days before discovery, release coordination, and production validation.

Coupling and Complexity Signals

	S	M	L	XL
CRITICAL	0	0	0	0
HIGH	0	0	6	0
MEDIUM	0	1	1	0
LOW	2	2	0	0

Technical Debt Register

This register groups evidence by engineering risk domain. Each row connects an observed signal to its likely change-risk implication and a concrete next action; it does not assign financial impact.

DEBT CATEGORY	SIGNAL	SEVERITY	EVIDENCE	ENGINEERING IMPACT	RECOMMENDED ACTION
Maintainability debt	71 / 100	medium	11 maintainability finding(s). 46.2 p95 complexity. 34.4% large-unit share.	Higher maintenance cost, slower onboarding, and greater regression risk during feature work.	Prioritize high-risk functions and large files, then convert repetitive patterns into named helpers.
Ownership concentration debt	90% top contributor share	medium	5 contributor(s). 62 recent commit(s). 15 git hotspot(s). Ownership inference threshold met.	Knowledge concentration creates key-person risk during handoff and enterprise support.	Document ownership, rotate reviews, and move implicit operational knowledge into runbooks.
Security debt	100 / 100	low	0 security finding(s). 0 dependency vulnerability signal(s). 0 fixable by package-manager workflow.	Security findings increase exploitability uncertainty, patch urgency, and release-review effort.	Resolve critical/high issues first, preserve audit output, and document residual accepted risk.
Architecture debt	100 / 100	low	15 components. 2 dependency edges. 0 cycle(s).	Coupling increases integration risk, test surface, and cost of changing core workflows.	Break cyclic dependencies, isolate cross-cutting modules, and define ownership boundaries.
Dependency governance debt	42 dependencies	low	0 risky version spec(s). 0 manifest(s) without lockfile. 0 missing license signal(s).	Unclear dependency provenance weakens technical confidence and increases supply-chain exposure.	Pin production dependencies, keep lockfiles, record licenses, and enable dependency update automation.
Delivery and testing debt	39% test ratio	low	1 delivery finding(s). 0 reliability finding(s). 0 safe fix(es) currently available.	Weak verification coverage turns routine fixes into delivery-risking delays.	Add smoke tests for core workflows, then expand regression coverage around risky code paths.
Framework evidence gaps	88 / 100	low	6 strong framework signal(s). 0 partial signal(s). 0 evidence gap(s).	Missing repository evidence increases uncertainty during security, quality, and change-control review.	Attach evidence to each mapped theme, close material gaps, and keep repository policies versioned.

Remediation Validation

Validation Scope

The post-remediation commit was exercised across the Python service, React/TypeScript desktop surface, Rust/Tauri runtime, production frontend build, dependency-audit paths, and the ClearCode scanner itself.

Executed Checks

LAYER	CHECK	RESULT
Python	uv run --group dev pytest -q	891 passed, 1 skipped
Frontend	npm test -- --run	69 tests passed
Frontend	npm run build	Production build passed
Rust/Tauri	cargo fmt --check and cargo check	Both checks passed
Dependencies	npm audit and pip-audit through ClearCode	2 of 2 audit ecosystems completed; 0 known vulnerabilities reported
ClearCode	npm test and npm run audit	117 tests passed; production audit passed

Validation Boundaries

No production deployment, live traffic, or organization-specific control effectiveness was evaluated.

The desktop application was not packaged or runtime-tested on macOS in this environment.

The score and delta describe repository evidence at the named commits; they are not a financial forecast or security certification.

Test and audit results describe the exercised environments and commands. They do not prove absence of defects, production reliability, or exploitability.

Framework Evidence Mapping

ALIGNMENT INDEX
88/100
weighted repository evidence proxy

STRONG SIGNALS
6
80–100 evidence score

PARTIAL SIGNALS
0
60–79 evidence score

EVIDENCE GAPS
0
below 60 evidence score

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

FRAMEWORK THEME	CATEGORY	SIGNAL	REPOSITORY EVIDENCE	PRIORITY ACTION
ISO 9001 ISO 9001:2015 Repository Evidence Signals	Quality process	88/100	Automated tests are present.	Create CI/test gates
ISO 27001 ISO/IEC 27001 Repository Evidence Signals	Security management	100/100	Security disclosure policy is present.	Add security policy
ISO 25010 ISO/IEC 25010 Product-Quality Evidence Signals	Product quality	84/100	Architecture score is 100/100.	Reduce red-zone functions
ISO 42001 ISO/IEC 42001 AI-Governance Evidence Signals	AI governance	86/100	AI governance score is 100/100.	Require CI for AI changes
SOC 2 SOC 2 Repository Evidence Signals	Trust controls	86/100	Security policy evidence is present.	Collect security/change-control evidence
Safety High-Assurance Repository Evidence Signals	High assurance	84/100	Automated tests are present.	Add traceable verification gates

Prioritized Findings Ledger

Findings are ordered by severity, then confidence and applicability. Test, fixture, example, and documentation signals are labeled so they cannot silently inherit production-risk claims.

RISK	FINDING AND LOCATION	EVIDENCE QUALITY	REMEDIATION / VERIFICATION
HIGH L effort	Large file hotspot surfaces/gui/src/App.tsx:1 clearcode.maintainability.large-file-hotspot 1969 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/App.tsx:1 for the corrected pattern.
HIGH L effort	Large file hotspot coworker/server/app.py:1 clearcode.maintainability.large-file-hotspot 1748 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/server/app.py:1 for the corrected pattern.
HIGH L effort	Large file hotspot coworker/server/manager.py:1 clearcode.maintainability.large-file-hotspot 3490 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/server/manager.py:1 for the corrected pattern.
HIGH L effort	Function complexity hotspot: App surfaces/gui/src/App.tsx:196 clearcode.maintainability.function-complexity-hotspot-app App spans 1691 non-empty lines with 402 decision token(s) and a 75.1/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split App around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/App.tsx:196 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH L effort	Function complexity hotspot: Sidebar surfaces/gui/src/components/Sidebar.tsx:104 clearcode.maintainability.function-complexity-hotspot-sidebar Sidebar spans 1020 non-empty lines with 204 decision token(s) and a 69.9/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split Sidebar around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/components/Sidebar.tsx:104 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.
HIGH L effort	Function complexity hotspot: Composer surfaces/gui/src/components/Composer.tsx:82 clearcode.maintainability.function-complexity-hotspot-composer Composer spans 434 non-empty lines with 144 decision token(s) and a 60.8/100 modeled complexity score.	high confidence direct applicability production scope	Change: Split Composer around explicit policy, state, or I/O boundaries while preserving its public contract; add focused tests for each extracted path. Proof: Run the affected test suite and re-scan surfaces/gui/src/components/Composer.tsx:82 ; verify the function's decision count and modeled complexity both decrease without moving complexity into an untested helper.

Prioritized Findings Ledger

Continuation of the prioritized evidence ledger. This page contains findings 7–12 of 12 using the same severity, confidence, applicability, and production-scope ordering.

RISK	FINDING AND LOCATION	EVIDENCE QUALITY	REMEDIATION / VERIFICATION
MEDIUM M effort	Contributor concentration risk Repository clearcode.delivery.contributor-concentration-risk 90% of recent commits came from one contributor.	medium confidence review applicability repository scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and confirm the repository-level signal improves.
MEDIUM L effort	Large file hotspot surfaces/gui/src/components/Sidebar.tsx:1 clearcode.maintainability.large-file-hotspot 1117 non-empty lines make ownership, testing, and refactoring harder.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review surfaces/gui/src/components/Sidebar.tsx:1 for the corrected pattern.
LOW S effort	Duplicate implementation block tests/test_automation.py:299 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 5 repeated blocks across 1 component(s), increase change cost.	high confidence contextual applicability test scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review tests/test_automation.py:299 for the corrected pattern.
LOW M effort	Duplicate implementation block coworker/connectors/browser_automation.py:29 clearcode.maintainability.duplicate-implementation-block 3 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/connectors/browser_automation.py:29 for the corrected pattern.
LOW M effort	Duplicate implementation block coworker/providers/anthropic_provider.py:24 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence direct applicability production scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review coworker/providers/anthropic_provider.py:24 for the corrected pattern.
LOW S effort	Duplicate implementation block tests/test_message_source.py:25 clearcode.maintainability.duplicate-implementation-block 1 duplicate cluster(s), up to 3 repeated blocks across 1 component(s), increase change cost.	high confidence contextual applicability test scope	Change: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks. Proof: Re-run the scan and review tests/test_message_source.py:25 for the corrected pattern.

Code Evidence Snippets

Bounded excerpts preserve the exact source context behind representative priority findings. Rule ID, confidence, fingerprint, remediation, and verification remain attached to each excerpt.

Large file hotspot

L effort

surfaces/gui/src/App.tsx:1 | Maintainability | high
clearcode.maintainability.large-file-hotspot | confidence high | fingerprint 4578e657c1fb

```
1 import {  
2   useCallback,
```

1969 non-empty lines make ownership, testing, and refactoring harder.

Remediation: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.

Verification: Re-run the scan and review surfaces/gui/src/App.tsx:1 for the corrected pattern.

Large file hotspot

L effort

coworker/server/app.py:1 | Maintainability | high
clearcode.maintainability.large-file-hotspot | confidence high | fingerprint d77a8601f4f1

```
1 """FastAPI app – OpenAI-compatible endpoint + WS session API + REST.  
2
```

1748 non-empty lines make ownership, testing, and refactoring harder.

Remediation: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.

Verification: Re-run the scan and review coworker/server/app.py:1 for the corrected pattern.

Large file hotspot

L effort

coworker/server/manager.py:1 | Maintainability | high
clearcode.maintainability.large-file-hotspot | confidence high | fingerprint 86cc50927642

```
1 """Session manager – owns engines (one per session), stores, and the provider.  
2
```

3490 non-empty lines make ownership, testing, and refactoring harder.

Remediation: Refactor the hotspot into a smaller, named unit and verify behavior with focused tests or checks.

Verification: Re-run the scan and review coworker/server/manager.py:1 for the corrected pattern.

Framework Evidence Appendix

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

ISO 9001:2015 Repository Evidence Signals

Evidence that the repository can be changed through repeatable, governed engineering process.

Evidence

Automated tests are present.
CI workflow is present.
Contribution workflow is documented.
Review ownership is explicit.

Gaps / Actions

6 critical/high finding(s) require management.
Create CI/test gates
Document contribution workflow

ISO/IEC 27001 Repository Evidence Signals

Repository-level security governance, vulnerability management, and exposure control signals.

Evidence

Security disclosure policy is present.
Dependency update automation is present.
No critical/high dependency vulnerabilities were detected.
Dependency audit returned no vulnerability entries.

Gaps / Actions

Add security policy
Enable dependency automation

ISO/IEC 25010 Product-Quality Evidence Signals

Software quality model coverage across maintainability, reliability, security, compatibility, and operability proxies.

Evidence

Architecture score is 100/100.
Security score is 100/100.
Tests support reliability validation.
CI supports repeatable verification.

Gaps / Actions

Maintainability score is 71/100.
Reduce red-zone functions
Strengthen test and CI evidence

Framework Evidence Appendix

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

ISO/IEC 42001 AI-Governance Evidence Signals

Controls for AI-assisted repository changes: traceability, human oversight, security, and release discipline.

Evidence

- AI governance score is 100/100.
- 100% traceable finding evidence.
- Security disclosure path exists for AI-assisted changes.
- CI can verify AI-assisted changes.

Gaps / Actions

- 6 critical/high finding(s) need human review.
- Require CI for AI changes
- Keep human ownership explicit

SOC 2 Repository Evidence Signals

Cloud trust readiness proxy for security, availability, change control, and evidence collection.

Evidence

- Security policy evidence is present.
- Change verification evidence is present.
- Approval ownership evidence is present.
- Dependency monitoring evidence is present.

Gaps / Actions

- Top contributor share is 90%, creating continuity risk.
- Collect security/change-control evidence
- Add owner approval gates

High-Assurance Repository Evidence Signals

Advisory proxy for stricter environments such as aviation, automotive, medical, and embedded software.

Evidence

- Automated tests are present.
- CI verification is present.
- No dependency cycles detected.
- Critical complex unit share is 4.2%.

Gaps / Actions

- 6 critical/high finding(s) remain.
- Add traceable verification gates
- Lower function complexity

Framework Sources and Interpretation Guide

Claim Boundary

These scores map visible repository artifacts to selected framework themes. They do not establish organizational process maturity, operating effectiveness, legal compliance, certification readiness, or audit assurance.

A score is a prioritization aid, not a substitute for threat modeling, runtime testing, architectural review, or audit evidence from organizational processes.

Research Basis

PRIMARY SOURCE	HOW IT INFORMS THIS REPORT	URL
NIST Secure Software Development Framework (SP 800-218)	Secure development practice and verification framing	https://csrc.nist.gov/pubs/sp/800/218/final
OpenSSF Scorecard	Repository security heuristics and explicit false-positive caveats	https://github.com/ossf/scorecard
OWASP Software Assurance Maturity Model	Governance, design, implementation, verification, and operations evidence	https://owasp.org/www-project-samm/
NIST AI Risk Management Framework	Govern, map, measure, and manage framing for AI-enabled systems	https://www.nist.gov/itl/ai-risk-management-framework
OWASP Top 10 for Agentic Applications 2026	Agent execution, tool, identity, memory, and human-oversight threat framing	https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

Confidence Interpretation

100% of findings include rule, fingerprint, impact, and source or repository evidence.

2/2 applicable package-manager audit(s) completed successfully.

Git history includes 62 fetched commit(s).

High: direct, rule-backed evidence with a specific source or repository control.

Medium: a credible heuristic requiring applicability review.

Low: weak or indirect evidence; do not prioritize without manual confirmation.

Applicability Interpretation

Direct: production-code behavior or maintainability risk.

Control: CI, build, release, workflow, or repository-policy risk.

Review: applicability is unresolved and requires manual inspection.

Contextual / advisory: test, fixture, example, or documentation evidence; excluded from direct production-risk claims.